Leyes de conservación: teoría, métodos numéricos y aceleración en GPU

Sesión 2: Introducción al High Performance Computing (HPC) y paralelización en GPU

Dr. Víctor Osores Escalona
Departamento de Matemática, Física y Estadística,
Facultad de Ciencias Básicas,
Universidad Católica del Maule

Universidad Tecnológica de Panamá, 2 al 6 de marzo, 2026

Facultad de Ciencias Básicas

dm2a Doctorado en Modelamiento Matemático Aplicado

Contenido

1. High Performance Computing
   - ¿Qué es High Performance Computing (HPC)?
   - ¿Por qué HPC no es lo mismo que "computación normal"?
   - Hardware típico en HPC (visión general)
   - Software que suele acompañar al hardwares
2. CUDA
   - Introducción
   - Computo sobre tarjetas gráficas
   - Arquitectura CUDA
   - Escribiendo un Kernel
3. Numba
4. Ejemplos en CUDA
   - Producto Matriz-Vector
   - Máximo $|x|$ en GPU
5. Referencias

High Performance Computing

¿Qué es High Performance Computing (HPC)?

- **HPC** es el uso de **múltiples recursos computacionales en paralelo** para resolver problemas que serían **demasiado lentos** en un computador tradicional.
- En vez de "un computador más rápido", HPC suele ser:
  - **muchos núcleos** trabajando a la vez (CPU),
  - **muchas GPUs** procesando en paralelo,
  - y/o **muchos nodos** conectados en un clúster.
- Objetivo principal: **reducir tiempo de cómputo** y/o **aumentar el tamaño/precisión** de los modelos.

High Performance Computing

¿Por qué HPC no es lo mismo que "computación normal"?

Diferencia clave

En HPC, el foco está en **paralelizar** y **escalar** el cálculo: dividir el problema en partes y ejecutarlas simultáneamente.

- **Computación tradicional:** 1 máquina, pocos núcleos, ejecución principalmente secuencial.
- **HPC:** decenas/miles de núcleos y/o GPUs; coordinación y comunicación entre procesos.
- HPC requiere pensar en:
  - **rendimiento** (performance),
  - **uso eficiente de memoria**,
  - **comunicación** entre tareas (latencia/ancho de banda),
  - **balance de carga**.

High Performance Computing: Hardware típico en HPC (visión general)

- **CPU (muchos núcleos):** buena para control, lógica, y tareas con ramificación.
- **GPU (miles de hilos):** ideal para operaciones masivas y repetitivas (matrices, filtros, simulaciones).
- **Memoria RAM:** capacidad y velocidad; jerarquía de cachés importa mucho.
- **Almacenamiento:** SSD, y a nivel clúster, sistemas distribuidos.

Idea central

En HPC, **la comunicación y la memoria** pueden ser tan importantes como el cómputo.

High Performance Computing: Hardware típico en HPC (visión general)

- **CPU (muchos núcleos):** buena para control, lógica, y tareas con ramificación.
- **GPU (miles de hilos):** ideal para operaciones masivas y repetitivas (matrices, filtros, simulaciones).
- **Memoria RAM:** capacidad y velocidad; jerarquía de cachés importa mucho.
- **Almacenamiento:** SSD, y a nivel clúster, sistemas distribuidos.

Idea central

En HPC, **la comunicación y la memoria** pueden ser tan importantes como el cómputo.

High Performance Computing: Software que suele acompañar al hardwares

- **Paralelismo en CPU:** OpenMP / hilos / vectorización SIMD.
- **Paralelismo distribuido:** MPI (Message Passing Interface) para comunicación entre nodos.
- **Aceleración con GPU:** CUDA, OpenCL, o bibliotecas (BLAS/cuBLAS, etc.).
- **Planificación de trabajos:** gestores de colas (p.ej., SLURM) para asignar recursos.

Mensaje

HPC no es solo "más hardware": es un **ecosistema** (hardware + software + buenas prácticas).

High Performance Computing: Software que suele acompañar al hardwares

- **Paralelismo en CPU:** OpenMP / hilos / vectorización SIMD.
- **Paralelismo distribuido:** MPI (Message Passing Interface) para comunicación entre nodos.
- **Aceleración con GPU:** CUDA, OpenCL, o bibliotecas (BLAS/cuBLAS, etc.).
- **Planificación de trabajos:** gestores de colas (p.ej., SLURM) para asignar recursos.

Mensaje

HPC no es solo "más hardware": es un **ecosistema** (hardware + software + buenas prácticas).

¿Por qué computación en GPU?

Muchos problemas científicos requieren millones de operaciones similares.

- Ciencia y simulación: clima, oceanografía, geofísica, astrofísica, materiales.
- Ingeniería: CFD, métodos numéricos (dif. finitas, volumenes finitos, elementos finitos), optimización de diseños.
- Datos e IA: entrenamiento e inferencia a gran escala (deep learning, modelos fundacionales).
- En CPU: ejecución mayoritariamente secuencial
- En GPU: miles de núcleos ejecutando en paralelo

CPU

- Pocos núcleos
- Muy complejos
- Optimizados para control y lógica

GPU

- Miles de núcleos simples
- Optimizados para paralelismo masivo
- Altísimo rendimiento en cómputo numérico

CUDA es útil cuando tenemos muchas operaciones iguales sobre muchos datos.

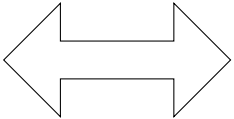
CUDA: Introducción

" CUDA® es una plataforma de cálculo paralelo y un modelo de programación desarrollado por NVIDIA para el cálculo general en unidades de procesamiento gráfico (GPU). Con CUDA, los desarrolladores pueden acelerar drásticamente las aplicaciones de cálculo aprovechando la potencia de las GPU."

Extraído de la web de Nvidia https://developer.nvidia.com/cuda-zone

CUDA: Computo sobre tarjetas gráficas

- Podemos definir la computación sobre tarjetas gráficas como el uso de una tarjeta gráfica (GPU) para realizar cálculos de propósito general.
- El modelo de computación sobre tarjetas gráficas consiste en usar conjuntamente una CPU y una GPU de manera que formen un modelo de computación heterogéneo.
- La parte secuencial de un programa se ejecutará sobre la CPU (Host) y la parte paralelizable del cálculo se ejecutará sobre la GPU (device).

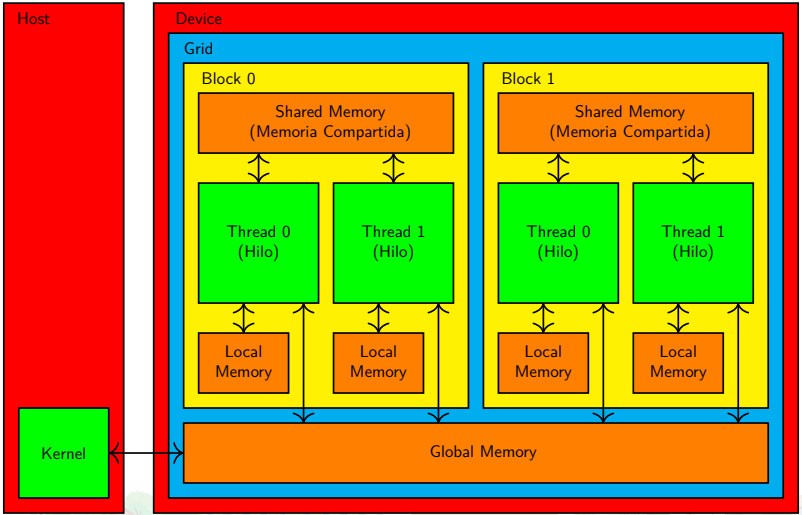


CPU (Host) GPU (Device)

CUDA: Arquitectura CUDA

- Inicialmente, las GPU estaban diseñadas principalmente para el procesamiento de gráficos.
- Para usarlas en cómputo general, los problemas debían reformularse para encajar en el pipeline gráfico (por ejemplo, mediante shaders y texturas).
- Al reconocer el potencial de esta capacidad de cómputo, NVIDIA lanzó CUDA (Compute Unified Device Architecture) hacia 2007, facilitando el acceso al paralelismo masivo de las GPU para aplicaciones científicas y de ingeniería.

CUDA: Arquitectura CUDA



Arquitectura de CUDA

Thread (Hilo): Unidad más elemental de procesamiento. Cada thread puede ejecutar una tarea independientemente del resto. Una GPU de sobremesa puede contener desde cientos a miles hilos.

Arquitectura de CUDA

Thread (Hilo): Unidad más elemental de procesamiento. Cada thread puede ejecutar una tarea independientemente del resto. Una GPU de sobremesa puede contener desde cientos a miles hilos.

Local Memory: Cada thread tiene asignada una memoria de acceso exclusivo llamada local memory (memoria local).

Arquitectura de CUDA

Thread (Hilo): Unidad más elemental de procesamiento. Cada thread puede ejecutar una tarea independientemente del resto. Una GPU de sobremesa puede contener desde cientos a miles hilos.

Local Memory: Cada thread tiene asignada una memoria de acceso exclusivo llamada local memory (memoria local).

Block (Bloque): Un Block (Bloque) es un arreglo de hilos de a lo más 1024 hilos.

Arquitectura de CUDA

Thread (Hilo): Unidad más elemental de procesamiento. Cada thread puede ejecutar una tarea independientemente del resto. Una GPU de sobremesa puede contener desde cientos a miles hilos.

Local Memory: Cada thread tiene asignada una memoria de acceso exclusivo llamada local memory (memoria local).

Block (Bloque): Un Block (Bloque) es un arreglo de hilos de a lo más 1024 hilos.

Share Memory: Cada bloque tiene asignada una memoria que es accesible por todos los hilos de ese bloque llamada Share Memory (memoria compartida). Es una memoria de rápida lectura y escritura y, es mayor capacidad que la memoria local. Los hilos pueden compartir información a través de esta memoria.

Arquitectura de CUDA

Thread (Hilo): Unidad más elemental de procesamiento. Cada thread puede ejecutar una tarea independientemente del resto. Una GPU de sobremesa puede contener desde cientos a miles hilos.

Local Memory: Cada thread tiene asignada una memoria de acceso exclusivo llamada local memory (memoria local).

Block (Bloque): Un Block (Bloque) es un arreglo de hilos de a lo más 1024 hilos.

Share Memory: Cada bloque tiene asignada una memoria que es accesible por todos los hilos de ese bloque llamada Share Memory (memoria compartida). Es una memoria de rápida lectura y escritura y, es mayor capacidad que la memoria local. Los hilos pueden compartir información a través de esta memoria.

Grid: Un Grid (grilla) es un arreglo de bloques.

Arquitectura de CUDA

Thread (Hilo): Unidad más elemental de procesamiento. Cada thread puede ejecutar una tarea independientemente del resto. Una GPU de sobremesa puede contener desde cientos a miles hilos.

Local Memory: Cada thread tiene asignada una memoria de acceso exclusivo llamada local memory (memoria local).

Block (Bloque): Un Block (Bloque) es un arreglo de hilos de a lo más 1024 hilos.

Share Memory: Cada bloque tiene asignada una memoria que es accesible por todos los hilos de ese bloque llamada Share Memory (memoria compartida). Es una memoria de rápida lectura y escritura y, es mayor capacidad que la memoria local. Los hilos pueden compartir información a través de esta memoria.

Grid: Un Grid (grilla) es un arreglo de bloques.

Global Memory: La memoria global es una memoria a la cual tienen acceso todos los bloques de la misma grilla. En general es una memoria de lenta lectura y escritura y de mayor capacidad que la memoria compartida.

Arquitectura de CUDA

Thread (Hilo): Unidad más elemental de procesamiento. Cada thread puede ejecutar una tarea independientemente del resto. Una GPU de sobremesa puede contener desde cientos a miles hilos.

Local Memory: Cada thread tiene asignada una memoria de acceso exclusivo llamada local memory (memoria local).

Block (Bloque): Un Block (Bloque) es un arreglo de hilos de a lo más 1024 hilos.

Share Memory: Cada bloque tiene asignada una memoria que es accesible por todos los hilos de ese bloque llamada Share Memory (memoria compartida). Es una memoria de rápida lectura y escritura y, es mayor capacidad que la memoria local. Los hilos pueden compartir información a través de esta memoria.

Grid: Un Grid (grilla) es un arreglo de bloques.

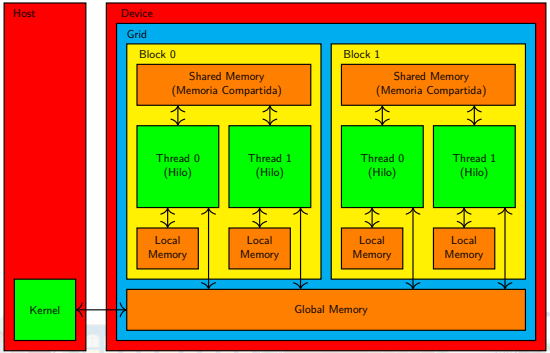
Global Memory: La memoria global es una memoria a la cual tienen acceso todos los bloques de la misma grilla. En general es una memoria de lenta lectura y escritura y de mayor capacidad que la memoria compartida.

Kernel: Es una secuencia de instrucciones (programa o algoritmo) que vienen desde el Host para que ejecute la el Device. Las instrucciones son las mismas para todos los hilos. Solo puede intercambiar datos con el Device a través de la memoria global. Además indica la cantidad de bloques e hilos por bloque que se van a utilizar.

CUDA

Ejercicio

- ¿Cuántos hilos se puede ejecutaren paralelo el dispositivo?
- ¿Cuántos hilos se puede comunicar rápido entre sí en el dispositivo ?
- ¿Cuántos hilos se puede comunicar entre sí en el dispositivo?



CUDA: resp.

¿Cuántos hilos se pueden ejecutar en paralelo?

Depende del hardware del dispositivo (número de SMs y recursos disponibles). Una GPU moderna puede ejecutar miles de hilos concurrentemente. Sin embargo, los hilos se ejecutan en grupos de 32 (warps) y se planifican dinámicamente.

¿Cuántos hilos pueden comunicarse rápido entre sí?

Solo los hilos que pertenecen al mismo bloque, ya que comparten Shared Memory. Un bloque puede tener hasta 1024 hilos, por lo tanto, como máximo 1024 hilos pueden comunicarse rápidamente entre sí.

¿Cuántos hilos pueden comunicarse entre sí en general?

Todos los hilos del Grid pueden comunicarse mediante Global Memory. Sin embargo, esta comunicación es más lenta y no existe sincronización directa entre bloques dentro de un mismo kernel.

Escribiendo un Kernel

CUDA basa su sintaxis en C++. Por lo que los Kernel tienen dicha sintaxis.

Ejemplo

Un ejemplo de un Kernel que suma, elemento a elemento, dos vectores a y b de tamaño n y almacena la respuesta en el vector c sería el siguiente:

```
#include <iostream>
#include <cuda_runtime.h>

// -------------------------
// Kernel (Device)
// -------------------------

__global__ void vecAdd(float *a, float *b, float *c, int n)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x; // índice global

    if (i < n)              // guard de seguridad
      c[i] = a[i] + b[i]; // cada hilo suma un elemento
}
```

¿Qué es Numba?

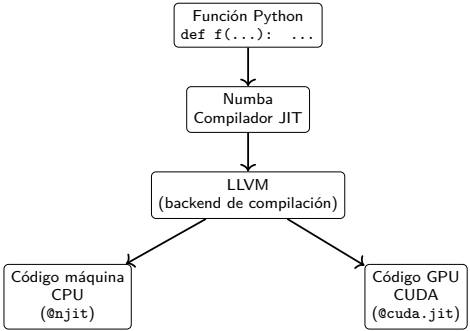
- Compilador JIT (Just-In-Time) para Python
- Traduce funciones a código máquina optimizado
- Soporta ejecución en CPU y GPU

Ventaja: escribir CUDA sin escribir CUDA C.

Numba: CPU vs GPU

- `@njit`: ejecución optimizada en CPU
- `@cuda.jit`: ejecución en GPU

En GPU solo se permite un subconjunto de Python.



Primer kernel CUDA vs Python puro

CUDA (Numba)

```
@cuda.jit
def add_kernel(a, b, c):
    i = cuda.grid(1)
    if i < c.size:
        c[i] = a[i] + b[i]
```

Cada thread calcula una componente del vector.

Python puro (CPU)

```
def add_python(a, b, c):
    for i in range(len(c)):
        c[i] = a[i] + b[i]
```

Un solo hilo recorre el vector secuencialmente.

Note que el cuerpo del cálculo es el mismo, lo que cambia es quién ejecuta el loop (CPU secuencial vs GPU paralela).

Relación con Numba CUDA (modelo mental ↔ código)

- `Host` → Python (CPU): prepara datos y lanza kernels
- `Device` → GPU: ejecuta kernels sobre datos en memoria global
- `Kernel` → función con `@cuda.jit`
- `Thread` → índice global con `cuda.grid(1)`
- `Grid` → `kernel[blocks, threads](...)`
- `Transferencias` → `to_device()` y `copy_to_host()`

El **kernel** corre muchas veces (una por thread) y conviene minimizar Host↔Device.

```python
import numpy as np
from numba import cuda
@cuda.jit
def add_kernel(a, b, c):
    i = cuda.grid(1)
    if i < c.size:
        c[i] = a[i] + b[i]
# ---- Host (CPU) ----
n = 1000000
a = np.ones(n, np.float32)
b = np.ones(n, np.float32)
# Host -> Device (copias a GPU)
d_a = cuda.to_device(a); d_b = cuda.to_device(b)
d_c = cuda.device_array_like(a)
threads = 256
blocks  = (n + threads - 1) // threads
# Grid launch: [blocks, threads]
add_kernel[blocks, threads](d_a, d_b, d_c)
cuda.synchronize()
# Device -> Host (resultado a CPU)
c = d_c.copy_to_host()
```

Relación con Numba CUDA (índices explícitos: Thread/Block/Grid)

- Thread → `cuda.threadIdx.x` (id dentro del bloque)
- Block → `cuda.blockIdx.x` (id del bloque)
- BlockDim → `cuda.blockDim.x` (threads por bloque)
- Índice global → `i = bx * bd + tx`
- Grid launch → `kernel[blocks, threads](...)`

```python
import numpy as np
from numba import cuda

@cuda.jit
def add_kernel(a, b, c):
    tx = cuda.threadIdx.x
    bx = cuda.blockIdx.x
    bd = cuda.blockDim.x

    i = bx * bd + tx
    if i < c.size:
        c[i] = a[i] + b[i]
```

Información del dispositivo (eje1.py)

```python
from numba import cuda

device = cuda.get_current_device()

print("name:", device.name)
print("compute_capability:", device.compute_capability)
print("MULTIPROCESSOR_COUNT:", device.MULTIPROCESSOR_COUNT)
print("MAX_THREADS_PER_BLOCK:", device.MAX_THREADS_PER_BLOCK)
print("MAX_BLOCK_DIM_X:", device.MAX_BLOCK_DIM_X)
print("MAX_BLOCK_DIM_Y:", device.MAX_BLOCK_DIM_Y)
print("MAX_BLOCK_DIM_Z:", device.MAX_BLOCK_DIM_Z)
print("MAX_GRID_DIM_X:", device.MAX_GRID_DIM_X)
print("MAX_GRID_DIM_Y:", device.MAX_GRID_DIM_Y)
print("MAX_GRID_DIM_Z:", device.MAX_GRID_DIM_Z)
print("WARP_SIZE:", device.WARP_SIZE)
print("MAX_SHARED_MEMORY_PER_BLOCK:", device.MAX_SHARED_MEMORY_PER_BLOCK)
print("MAX_REGISTERS_PER_BLOCK:", device.MAX_REGISTERS_PER_BLOCK)
print("CLOCK_RATE:", device.CLOCK_RATE)
```

Información del dispositivo

```python
from numba import cuda
# Obtiene el dispositivo CUDA "actual" (la GPU que Numba está usando en esta sesión).
device = cuda.get_current_device()

# -------------------------
# Identidad del dispositivo
# -------------------------
print("name:", device.name)
# name: nombre del modelo de GPU (ej. "NVIDIA GeForce RTX ...")

print("compute_capability:", device.compute_capability)
# compute_capability: (major, minor) = versión de la arquitectura CUDA (SM),
# útil para saber qué instrucciones/features soporta la GPU (p.ej. 7.5, 8.6, 9.0, etc.)

# -------------------------
# Paralelismo a nivel "hardware"
# -------------------------
print("MULTIPROCESSOR_COUNT:", device.MULTIPROCESSOR_COUNT)
# MULTIPROCESSOR_COUNT: cantidad de multiprocesadores (SMs).
# Cada SM ejecuta muchos hilos en paralelo; más SMs suele implicar más rendimiento.
```

Información del dispositivo

```
print("WARP_SIZE:", device.WARP_SIZE)
# WARP_SIZE: número de hilos que avanzan "en lockstep" (en CUDA, típicamente 32).

# -------------------------
# Límites de configuración (launch) de kernels
# -------------------------
print("MAX_THREADS_PER_BLOCK:", device.MAX_THREADS_PER_BLOCK)
# MAX_THREADS_PER_BLOCK: máximo total de threads permitidos en un bloque (p.ej. 1024).
# Nota: en 2D/3D igual aplica al total: blockDim.x * blockDim.y * blockDim.z <= este valor.

print("MAX_BLOCK_DIM_X:", device.MAX_BLOCK_DIM_X)
print("MAX_BLOCK_DIM_Y:", device.MAX_BLOCK_DIM_Y)
print("MAX_BLOCK_DIM_Z:", device.MAX_BLOCK_DIM_Z)
# MAX_BLOCK_DIM_*: límites por dimensión del bloque (blockDim.x, blockDim.y, blockDim.z).
# Te dice hasta qué tamaños puedes usar en cada eje del bloque.

print("MAX_GRID_DIM_X:", device.MAX_GRID_DIM_X)
print("MAX_GRID_DIM_Y:", device.MAX_GRID_DIM_Y)
print("MAX_GRID_DIM_Z:", device.MAX_GRID_DIM_Z)
# MAX_GRID_DIM_*: límites por dimensión de la grilla (gridDim.x, gridDim.y, gridDim.z).
# Te dice cuántos bloques como máximo puedes lanzar en cada eje.
```

Información del dispositivo

```
# -------------------------
# Recursos por bloque (lo que "cabe" en un SM al ejecutar un bloque)
# -------------------------
print("MAX_SHARED_MEMORY_PER_BLOCK:", device.MAX_SHARED_MEMORY_PER_BLOCK)
# MAX_SHARED_MEMORY_PER_BLOCK: cantidad máxima de memoria compartida (shared memory)
# disponible por bloque (en bytes). Limita cuánto puedes usar con cuda.shared.array(...)

print("MAX_REGISTERS_PER_BLOCK:", device.MAX_REGISTERS_PER_BLOCK)
# MAX_REGISTERS_PER_BLOCK: número máximo de registros disponibles por bloque.
# Más registros por thread puede mejorar velocidad, pero demasiados reduce ocupación (occupancy).

# -------------------------
# Reloj del dispositivo (orientativo)
# -------------------------
print("CLOCK_RATE:", device.CLOCK_RATE)
# CLOCK_RATE: frecuencia del reloj del GPU (normalmente en kHz).
# No es un predictor perfecto de rendimiento, pero sirve como referencia.
```

Interpretación del resultado

Enunciado (grilla 2D, bloques 2D)

Construya un kernel que, para cada hilo, imprima:

Hola Mundo!, soy el hilo (tx,ty) del bloque (bx,by), pero mi ID es (idx,idy) y mi IDG es IDG.

donde (tx, ty) es el hilo dentro del bloque, (bx, by) el bloque en la grilla, (idx, idy) la coordenada global del hilo e IDG un identificador global 1D.

Supongamos la configuración:

- `threads = (2,2)` $\Rightarrow$ cada bloque tiene 2×2 hilos
- `blocks = (2,2)` $\Rightarrow$ la grilla tiene 2×2 bloques

Entonces:

$$N_x = \texttt{gridDim.x} \cdot \texttt{blockDim.x} = 2 \times 2 = 4, \qquad N_y = \texttt{gridDim.y} \cdot \texttt{blockDim.y} = 2 \times 2 = 4.$$

Hay $4 \times 4 = 16$ hilos globales. Cada mensaje imprime:

- (tx, ty): posición del hilo dentro del bloque
- (bx, by): posición del bloque dentro de la grilla
- (idx, idy): coordenada global del hilo
- IDG: identificador global 1D

Cálculo del ID Global (grilla_2d.py)

Enunciado (definición de IDs)

- **ID por componente:** (idx, idy) debe ser único para cada hilo en la grilla.
- **ID global:** IDG debe ser un entero único para cada hilo.

El ID global 2D se calcula como,

$$idx = bx \cdot \texttt{cuda.blockDim.x} + tx, \qquad idy = by \cdot \texttt{cuda.blockDim.y} + ty.$$

El ID global 1D (linealizado) es,

$$IDG = idy \cdot N_x + idx, \qquad N_x = \texttt{cuda.gridDim.x} \cdot \texttt{cuda.blockDim.x}.$$

¿Por qué el orden de impresión parece desordenado?

Enunciado (observación)

Al ejecutar el kernel, la salida no necesariamente aparece ordenada por IDG.

Aunque el ID global esté bien definido, la salida no aparece en orden,

- La GPU ejecuta bloques en paralelo.
- Dentro de cada bloque, los hilos se ejecutan en **warps** (32 hilos).
- El `print()` en GPU no garantiza orden secuencial.

Idea clave

CUDA es paralelismo real. No existe un "for i = 0,1,2,3,..." como en CPU.

Consecuencia

El IDG puede ser correcto (0,1,2,3,...,15) pero la impresión puede verse como:

$$0, 1, 4, 5, 10, 11, 14, 15, 8, 9, 12, 13, 2, 3, 6, 7$$

Benchmark: CPU (Python) vs CPU (Numba) vs GPU (CUDA/Numba)

Objetivo: medir aceleración de GPU en un cómputo `compute-bound` (muchas FLOPs por elemento) y comparar contra CPU puro y CPU con Numba (paralelo + fastmath).

Idea del experimento

Para cada i (independiente):

$$(x, y) \leftarrow (a_i, b_i), \qquad \text{repetir } \texttt{iters} \text{ veces una actualización algebraica,} \qquad c_i \leftarrow x + y.$$

Esto es **paralelismo de datos** ideal: cada hilo/iteración trabaja su propio índice.

GPU (Numba CUDA): 1D grid

```
@cuda.jit
def heavy_gpu(a,b,c,iters):
    i = cuda.grid(1)
    if i < c.size:
        x = a[i]; y = b[i]
        for _ in range(iters):
            x = x*1.000001 + y*0.999999
            y = y*1.000003 - x*0.999997
        c[i] = x + y
```

CPU (Numba): paralelo en hilos CPU

```
@njit(parallel=True, fastmath=True)
def heavy_cpu_numba(a,b,c,iters):
    for i in prange(c.size):
        x=a[i]; y=b[i]
        for _ in range(iters):
            x = x*1.000001 + y*0.999999
            y = y*1.000003 - x*0.999997
        c[i] = x + y
```

Benchmark: CPU (Python) vs CPU (Numba) vs GPU (CUDA/Numba) `eje2.py`

Configuración de lanzamiento (GPU)

```
threads = 256
blocks  = (n + threads - 1) // threads   # ceil(n/threads)
heavy_gpu[blocks, threads](d_a, d_b, d_c, iters)
cuda.synchronize()
```

Qué se mide y cómo se valida

- **Tiempo GPU:** solo el kernel (sin transferencias Host↔Device). Se hace **warm-up** para compilar.
- **Tiempo CPU:** Numba (ya compilado) vs Python puro.
- **Speedup:** CPU/GPU.
- **Error:** $\max_i |c_{\text{cpu}} - c_{\text{gpu}}|$ (puede variar por `fastmath` y orden de operaciones).

Producto Matriz-Vector: CPU (Python) vs GPU (Numba CUDA)

Objetivo: comparar el cómputo $y = A\,x$ (cada fila de A hace un producto punto con x) en **CPU Python puro** vs **GPU** con un kernel 1D (un hilo por fila).

Estrategia (row-major): 1 hilo → 1 fila

Para cada fila i:

$$y_i = \sum_{j=0}^{n-1} A_{i,j}\, x_j.$$

Paralelismo: filas independientes ⇒ se paraleliza sobre i.

CPU (Python puro)

```python
def matvec_py(A, x, y):
    m = A.shape[0]; n = A.shape[1]
    for i in range(m):
        s = 0.0
        for j in range(n):
            s += A[i, j] * x[j]
        y[i] = s
```

GPU (Numba CUDA): 1D grid

```python
@cuda.jit
def matvec_rowmajor(A, x, y):
    i = cuda.grid(1)
    m = A.shape[0]; n = A.shape[1]
    if i < m:
        s = 0.0
        for j in range(n):
            s += A[i, j] * x[j]
        y[i] = s
```

Producto Matriz-Vector: CPU (Python) vs GPU (Numba CUDA) `eje3.py`

Lanzamiento del kernel (GPU)

```
threads = 256
blocks  = (m + threads - 1) // threads  # ceil(m/threads)
matvec_rowmajor[blocks, threads](d_A, d_x, d_y)
cuda.synchronize()
```

Interpretación: `i = blockIdx.x * blockDim.x + threadIdx.x`.
Cada hilo calcula exactamente **una** componente $y[i]$ (una fila).

Qué se mide y validación

- **Warm-up GPU:** primera llamada compila el kernel; no se usa para timing.
- **Timing GPU:** mide solo ejecución del kernel (se sincroniza con `cuda.synchronize()`).
- **Timing CPU:** Python puro (doble loop).
- **Speedup:** CPU/GPU; se imprimen primeros valores de y como verificación rápida.

Producto Matriz-Vector: CPU (Python) vs GPU (Numba CUDA)

Lanzamiento del kernel (GPU)

```
threads = 256
blocks  = (m + threads - 1) // threads  # ceil(m/threads)
matvec_rowmajor[blocks, threads](d_A, d_x, d_y)
cuda.synchronize()
```

Nota importante (memoria y tamaño)

Este ejemplo es **muy intensivo en memoria**: A ocupa $m \cdot n$ floats.
Por ejemplo, si $n = 100{,}000$, entonces A tendría 10^{10} entradas $\Rightarrow \approx 40$ GB en `float32`, lo que normalmente **no cabe** en RAM/GPU. En práctica se usa n mucho menor o matrices dispersas / por bloques.

Reducción paralela en GPU: $\max(|x|)$ (ver `max_comp.py`)

Queremos calcular:

$$\max\left(|x[start : start + ncount]|\right)$$

sin copiar el vector completo al CPU.

Estrategia:

- Dividimos el rango en **tiles** de tamaño $2 \cdot \texttt{tpb}$.
- Cada bloque procesa un tile.
- Cada hilo carga hasta **2 elementos**.
- Dentro del bloque se hace una **reducción en árbol**.
- Cada bloque escribe un máximo parcial en `out[bid]`.
- Repetimos el proceso hasta obtener un único valor.

Ventaja: sólo se transfiere **1 escalar** al host.

Ejemplo: $tpb = 8$, $ncount = 25$

Cada bloque procesa:

$$2 \cdot tpb = 16 \text{ elementos}$$

Número de bloques:

$$\left\lceil \frac{25}{16} \right\rceil = 2$$

Para el bloque $bid = 0$:

$$k = bid \cdot (2\,tpb) + tid$$

Con $tid = 0, \ldots, 7$:

$$k = 0, 1, \ldots, 7$$

y además

$$k_2 = k + tpb = 8, \ldots, 15$$

Cada hilo calcula:

$$m = \max\left(|x[start + k]|,\ |x[start + k_2]|\right)$$

Así, el bloque cubre los 16 primeros elementos del rango.

Reducción en árbol (shared memory)

Después de cargar los máximos locales,

$$sm[tid] = m.$$

Se ejecuta la reducción,

$$stride = \frac{tpb}{2}.$$

- stride $= 4$: Hilos `tid=0..3` hacen,

  `sm[0] = max(sm[0], sm[4])`, `sm[1] = max(sm[1], sm[5])`
  `sm[2] = max(sm[2], sm[6])`, `sm[3] = max(sm[3], sm[7])`.

- stride $= 2$: Hilos `tid=0..1` hacen,

  `sm[0] = max(sm[0], sm[2])`, `sm[1] = max(sm[1], sm[3])`.

- stride $= 1$: Hilo `tid=0`, hace,

  `sm[0] = max(sm[0], sm[1])`. (máximo del bloque)

El hilo $tid = 0$ escribe:

$$out[bid] = sm[0]$$

Luego se repite el proceso sobre `out` hasta quedar un único valor.

Preguntas para pensar: ¿$\max(|x|)$, $\min(|x|)$ o suma?

Pregunta 1

La estrategia anterior (tiles + shared memory + reducción en árbol) sirve también para calcular:

$$\min\left(|x[start : start + ncount]|\right) \quad \text{o} \quad \sum_{k=0}^{ncount-1} x[start + k] \ ?$$

Pregunta 2: ¿Cómo lo harían? (pistas)

- **Para** $\min(|x|)$**:** ¿qué cambio harías en el operador de reducción? ¿con qué valor inicial (identidad) deben partir los hilos fuera de rango?
- **Para la suma:** ¿qué cambio harías en la reducción en shared memory? ¿qué problemas de **precisión** aparecen al sumar en paralelo (`float32`)?

Idea clave: mantener el mismo patrón de paralelismo (tiles + reducción), pero cambiar la **operación** (`max` → `min` o +) y el **valor neutro**.

¿Y cómo calcular $\max(x)$ y $\min(x)$?

Hasta ahora hemos calculado

$$\max(|x|)$$

porque esta cantidad aparece frecuentemente en los **métodos numéricos** estudiados (por ejemplo en estimaciones de estabilidad o criterios tipo CFL).

Pero muchas aplicaciones requieren también calcular: $\max(x)$, $\min(x)$.

Observación:
La estrategia paralela es exactamente la misma:

- dividir el vector en **tiles**
- cada bloque procesa un tile
- usar **shared memory**
- aplicar una **reducción en árbol**

La única diferencia es el **operador de reducción**: $\max(a, b)$ o $\min(a, b)$ en lugar de $\max(|a|, |b|)$.
Es decir, basta reemplazar $|x[i]| \to x[i]$ y usar el operador correspondiente.
Pregunta: ¿qué elección debemos hacer para las **semillas iniciales**?

References

- S. Böhm. How to Optimize a CUDA Matrix Multiplication Kernel. Personal technical blog, 2022. Available at: https://siboehm.com/articles/22/CUDA-MMM
- NVIDIA Corporation. cuBLAS Library User Guide. NVIDIA CUDA Toolkit Documentation. Available at: https://docs.nvidia.com/cuda/cublas/index.html
- M. Harris. Optimizing Parallel Reduction in CUDA. NVIDIA Developer Technology, 2007. Available at: https://developer.download.nvidia.com/assets/cuda/files/reduction.pdf
- V. Volkov and J. Demmel. Benchmarking GPUs to tune dense linear algebra. Proceedings of SC08: International Conference for High Performance Computing, Networking, Storage and Analysis, 2008.
- NVIDIA Corporation. CUDA C Programming Guide. NVIDIA CUDA Toolkit Documentation. Available at: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html

Agradecimientos

Gracias por su atención

Quedo atento a sus preguntas y comentarios.